

Motion Planning Lecture 10

Optimization-Based Motion Planning

Wolfgang Hönig (TU Berlin) and Andreas Orthey (Realtime Robotics)

June 18, 2025

Recap Last Week

Last Week

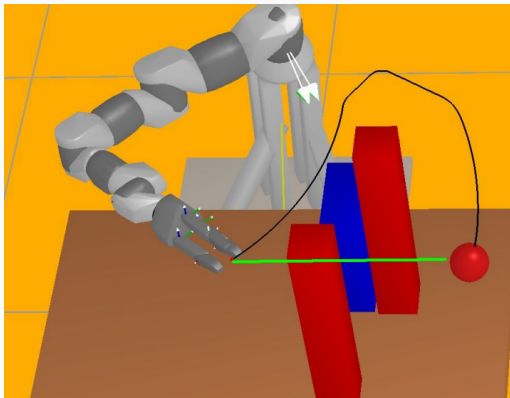
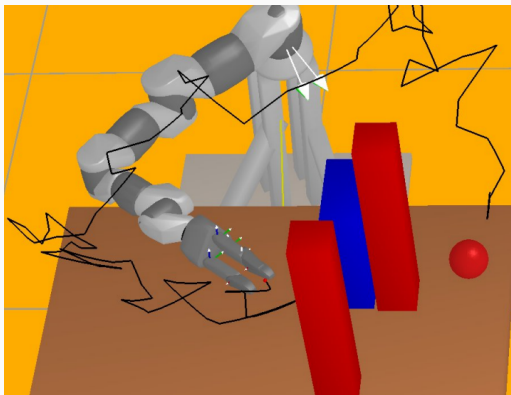
- Completeness and Optimality of Sampling-Based Planners
- Advanced sampling-based planners (EST, RRTConnect, PRM*, LazyPRM, FMT*)
- Introduction to Convex Optimization

Today

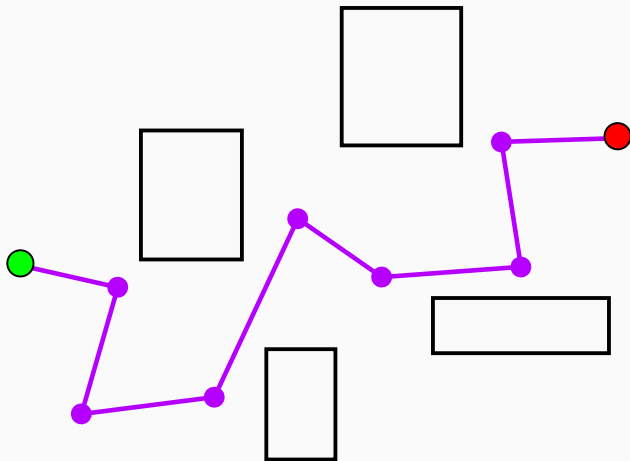
Optimization-based Motion Planning

- Optimization using Shortcutting and Splines
- Gradient-Based Optimization
- Efficient Gradients using Signed-Distance Fields

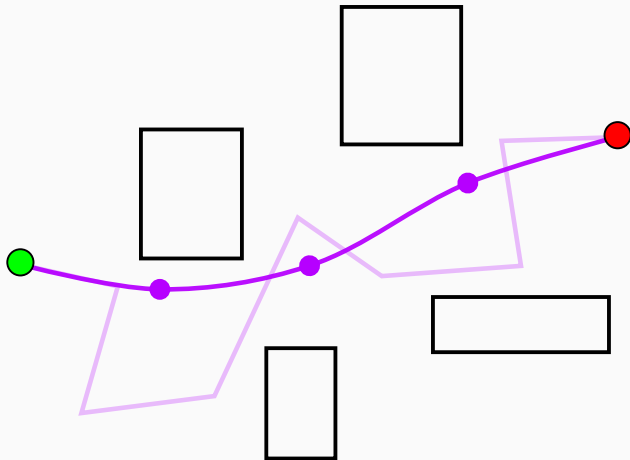
Optimization-based Motion Planning



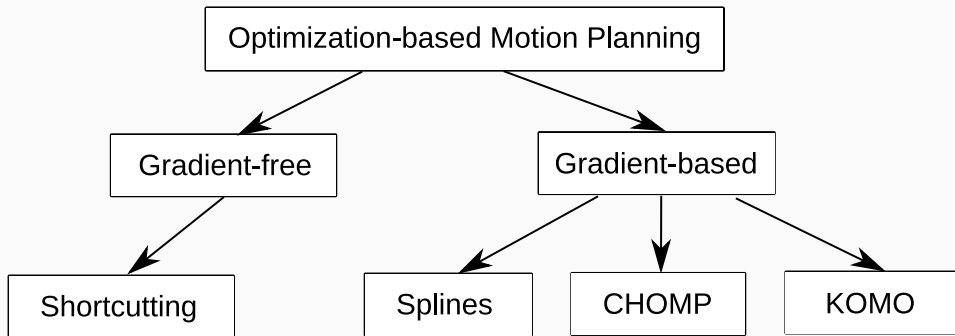
Optimization-based Motion Planning



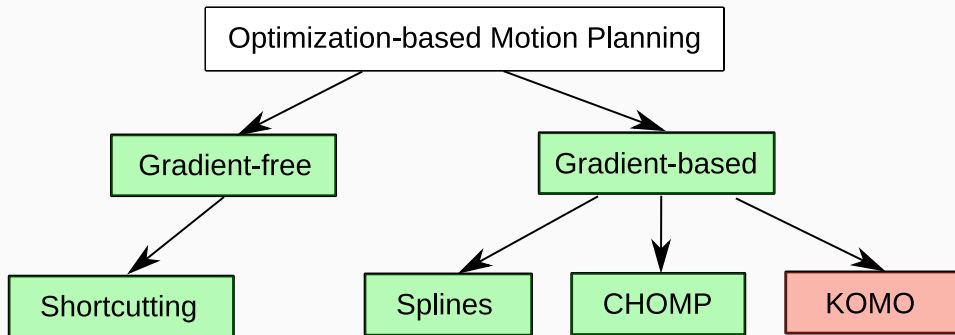
Optimization-based Motion Planning



Optimization-based Motion Planning



Optimization-based Motion Planning



Optimization using Shortcutting

Shortcutting

- Vanilla version
 - Sample two waypoints
 - Try to connect them and update path
 - Repeat until timeout or convergence
- Easy to implement
- Too few waypoints:
- Too many waypoints:

Shortcutting

- Vanilla version
 - Sample two waypoints
 - Try to connect them and update path
 - Repeat until timeout or convergence
- Easy to implement
- Too few waypoints: **Hard to connect**
- Too many waypoints:

Shortcutting

- Vanilla version
 - Sample two waypoints
 - Try to connect them and update path
 - Repeat until timeout or convergence
- Easy to implement
- Too few waypoints: **Hard to connect**
- Too many waypoints: **Long computational time**

Geometric Path Improvement

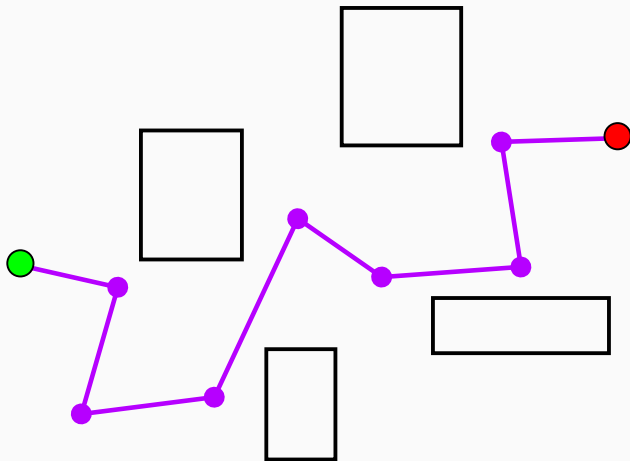
Shortcutting

- Vanilla version
 - Sample two waypoints
 - Try to connect them and update path
 - Repeat until timeout or convergence

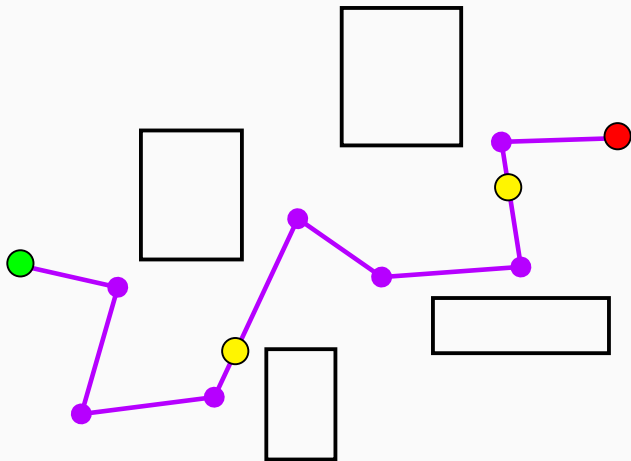
- Easy to implement
- Too few waypoints: **Hard to connect**
- Too many waypoints: **Long computational time**

How would you define convergence?

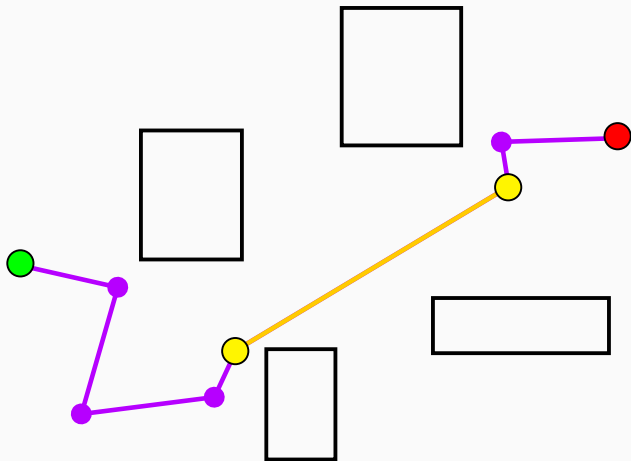
Shortcutting



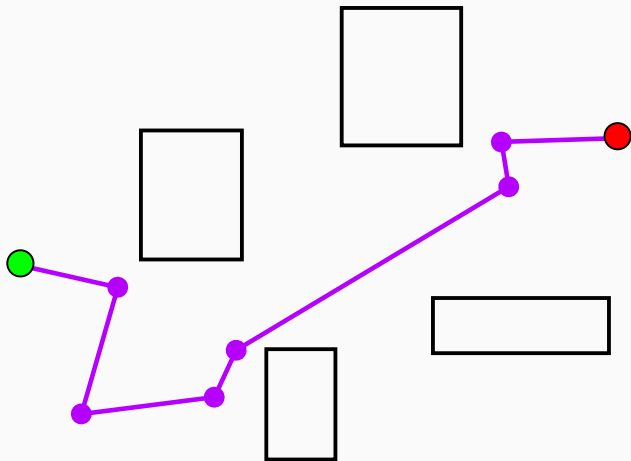
Shortcutting



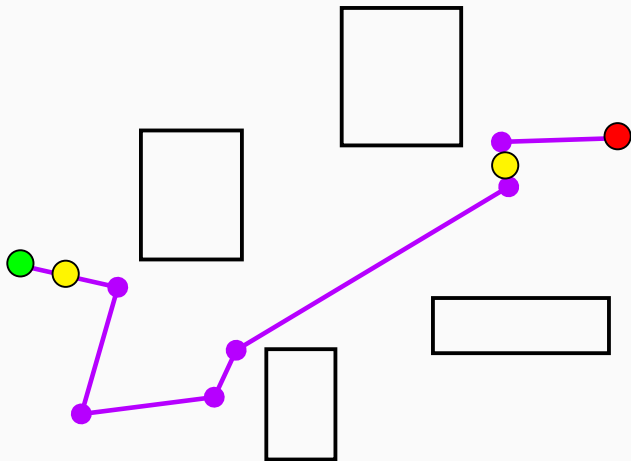
Shortcutting



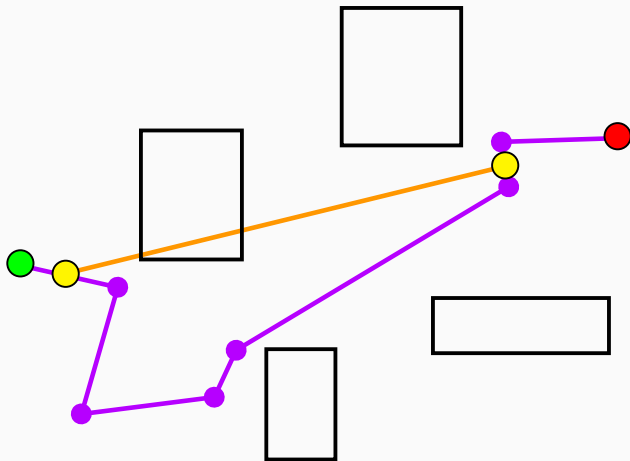
Shortcutting



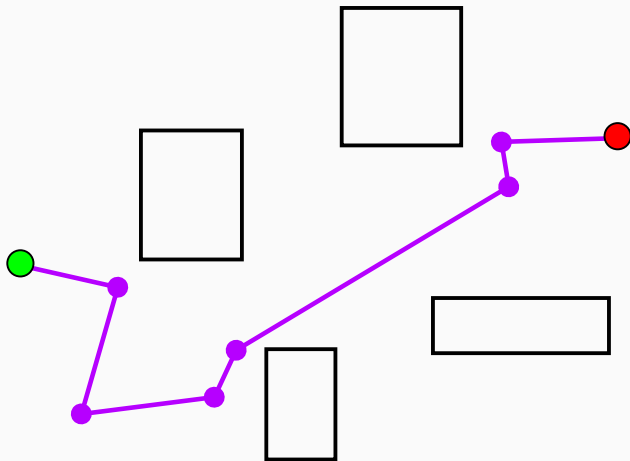
Shortcutting



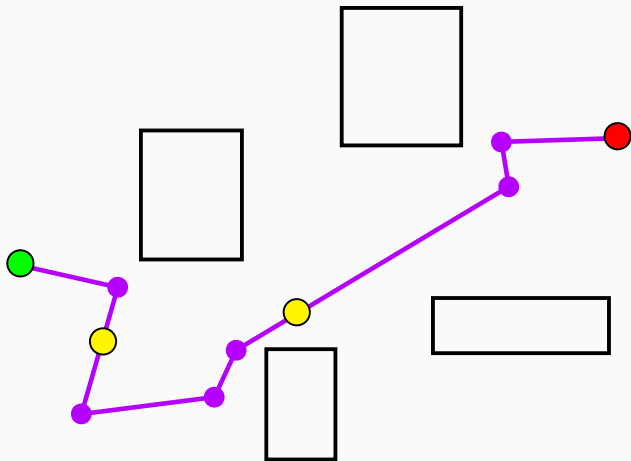
Shortcutting

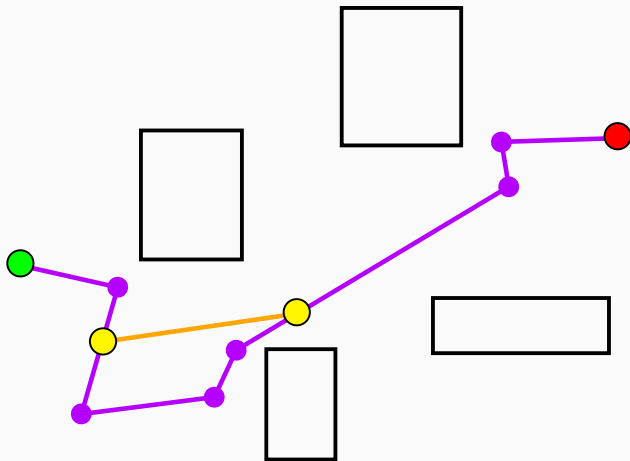


Shortcutting

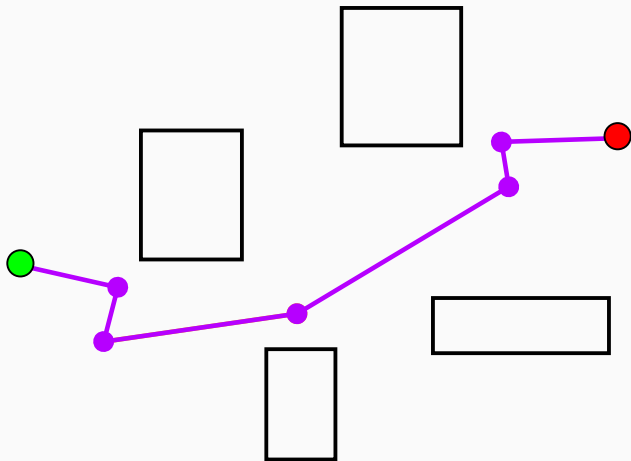


Shortcutting

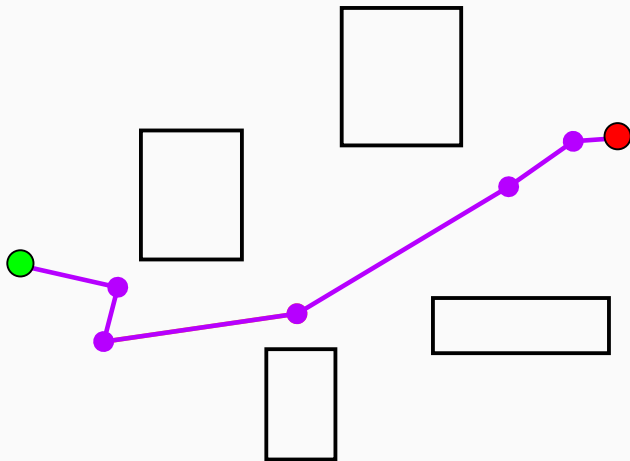


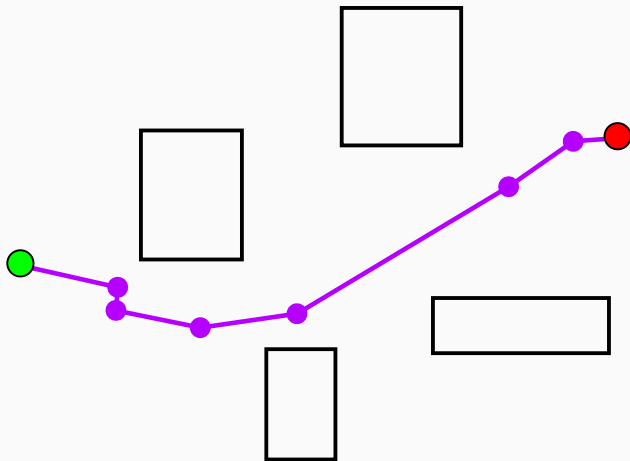


Shortcutting

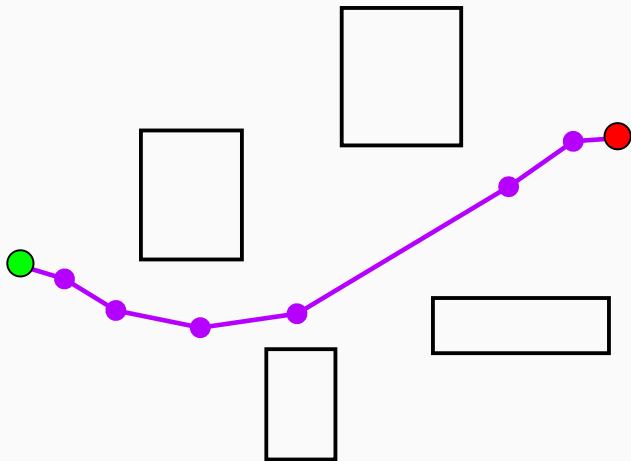


Shortcutting





Shortcutting



Shortcutting

- Question: Can you do shortcutting with an arbitrary cost function ?

Shortcutting

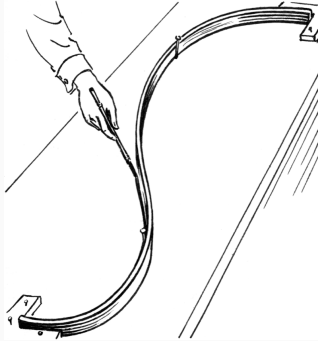
- Question: Can you do shortcutting with an arbitrary cost function ?
- Question: Can you do shortcutting on e.g. a sphere ?

Shortcutting

- Question: Can you do shortcutting with an arbitrary cost function ?
- Question: Can you do shortcutting on e.g. a sphere ?
- Question: Can you do shortcutting on kinodynamic systems ?

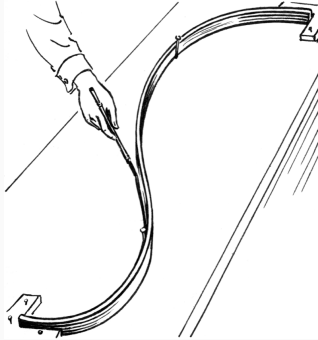
Optimization using Splines

Splines

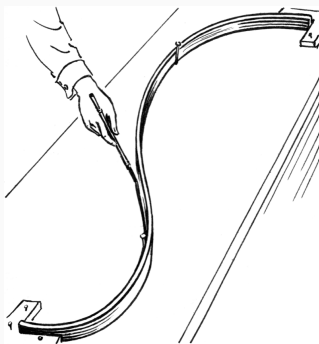


spline (n.)—long, thin piece of wood [Online Etymology Dictionary]

Splines



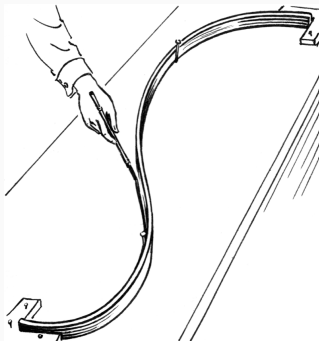
Mathematically: A piecewise polynomial function.



Mathematically: A piecewise polynomial function.

Why do we want that?

Splines

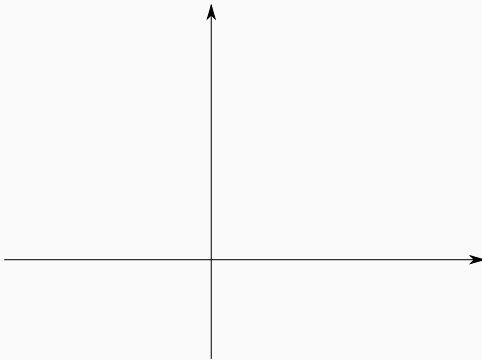


Mathematically: A piecewise polynomial function.

Why do we want that? Smoothness, Differentiability, Comfort (car)

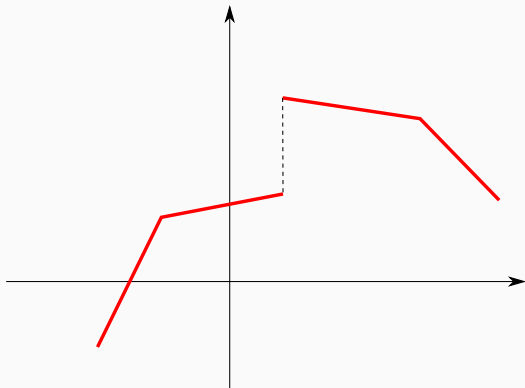
Smoothness

A path which is at least n -times differentiable (differentiability classes $C^0, C^1, \dots, C^\infty$).



Smoothness

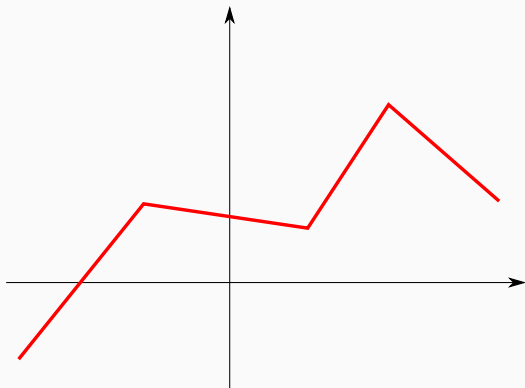
A path which is at least n -times differentiable (differentiability classes $C^0, C^1, \dots, C^\infty$).



Discontinuous function

Smoothness

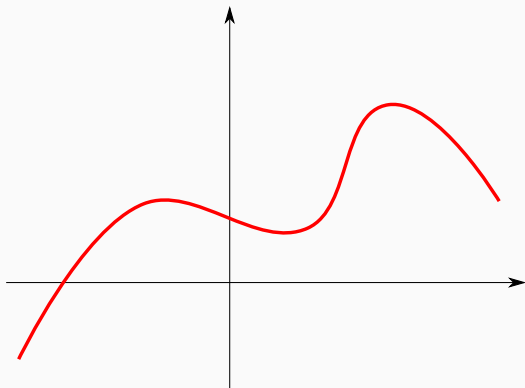
A path which is at least n -times differentiable (differentiability classes $C^0, C^1, \dots, C^\infty$).



C^0 function

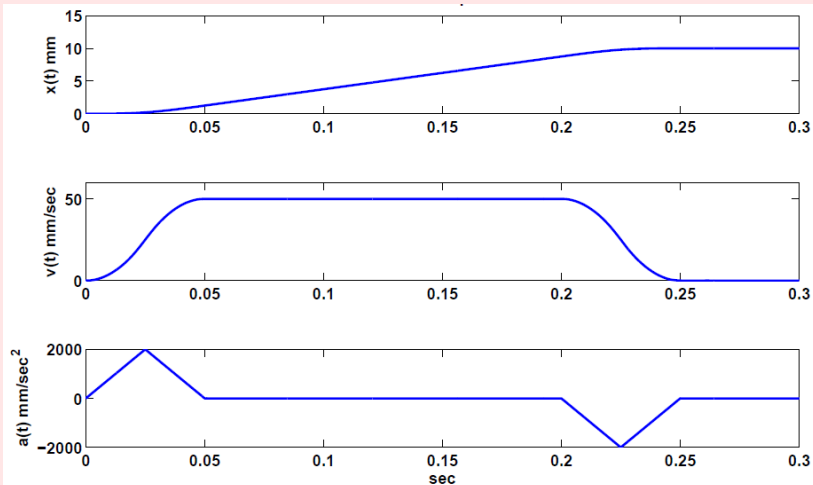
Smoothness

A path which is at least n -times differentiable (differentiability classes $C^0, C^1, \dots, C^\infty$).



C^1 function

Splines



Outline

- Basis splines (B-Splines)
- Polynomial splines
- Geometric planning with polynomial splines
- Bézier curves
- Safe planning with Bézier curves

Optimization using Splines

B-Splines

Basis Splines (BSplines)

- A path is represented by M control points p_m and corresponding basis functions

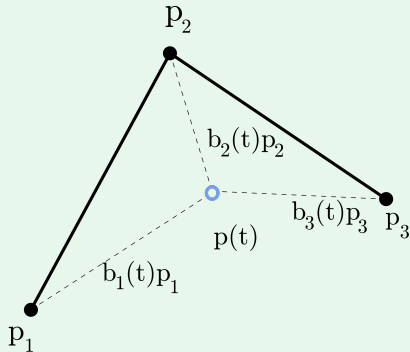
$$p(t) = \sum_{m=1}^M B_m(t)p_m, \text{ s.t. } \sum_{m=1}^M B_m(t) = 1 \text{ for all } t$$

- Interpretation basis function: Influence of control point p_m at point t .

Blending of basis functions

$$p(t) = \sum_{m=1}^M B_m(t) p_m$$

$$\text{s.t. } \sum_{m=1}^M B_m(t) = 1$$



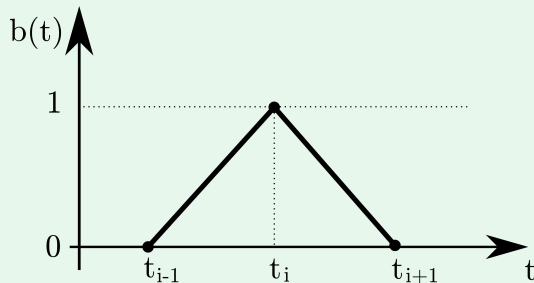
Knot vector

Let $p : [0, 1] \rightarrow X$ be a path.

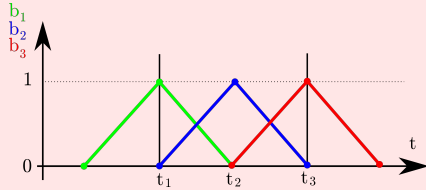
- Idea: Take the input space $[0, 1]$ and cover it with M subintervals $[t_i, t_{i+1}]$, $i = 0, \dots, M$, and with $t_i < t_{i+1}$.
- Step 2: For each t_i define a basis function centered at t_i .
- t_i is called the i -th knot, and $t = (t_0, \dots, t_M)$ the knot vector ($M + 1$ elements).

Linear B-Splines

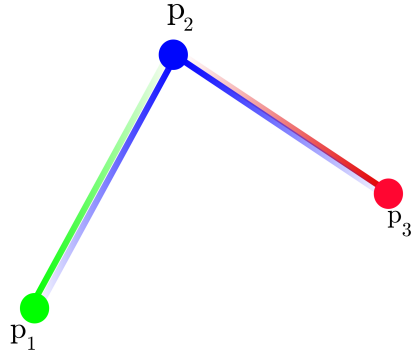
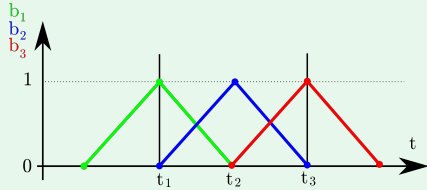
$$b_i(t) = \begin{cases} \frac{t - t_{i-1}}{t_i - t_{i-1}} & , \text{ if } t_{i+1} < t \leq t_i \\ \frac{-t + t_{i+1}}{t_{i+1} - t_i} & , \text{ if } t_i < t \leq t_{i+1} \\ 0 & , \text{ otherwise.} \end{cases}$$



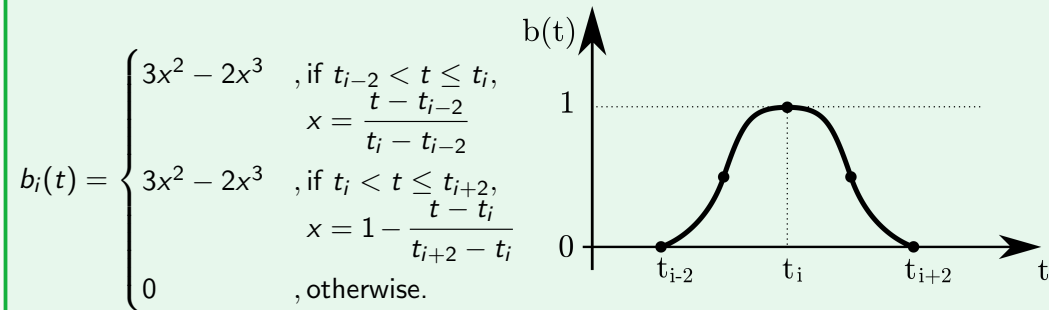
Linear B-Splines



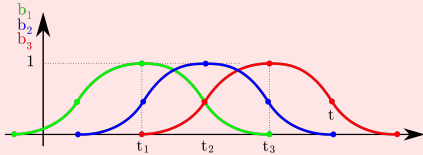
Linear B-Splines



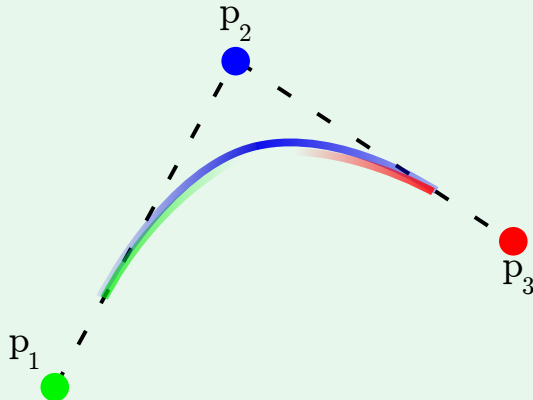
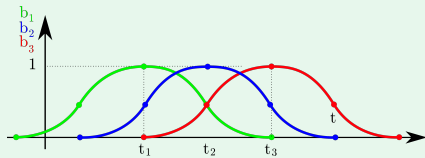
Hermit B-Splines



Hermit B-Splines



Hermit B-Splines



Cox-de Bour Recurrence

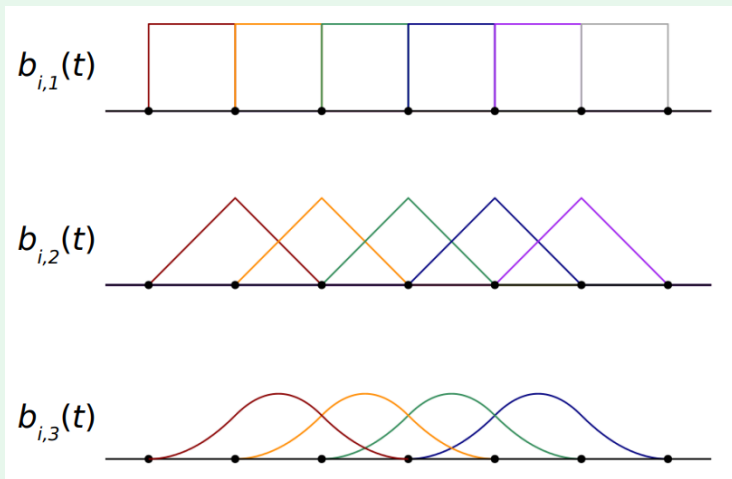
Recursive basis function (allows you to tune dimensionality and smoothness).

Choose an integer k . Then

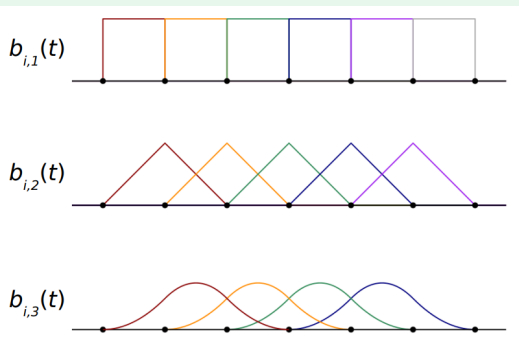
$$b_{i,1}(t) = \begin{cases} 1 & , \text{ if } t_i \leq t < t_{i+1} \\ 0 & , \text{ otherwise.} \end{cases}$$

$$b_{i,k}(t) = \frac{t - t_i}{t_{i+k} - t_i} b_{i,k-1}(t) + \frac{t_{i+k+1} - t}{t_{i+k+1} - t_{i+1}} b_{i+1,k-1}(t)$$

Cox-de Bour Recurrence



Cox-de Bour Recurrence



In which differentiability class is the Cox-de Bour recurrence function $b_{i,n}$? (i.e. $C^0, C^1, C^2, \dots$)

Optimization using Splines

Polynomial Splines

Polynomial

$$p(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n = \sum_{k=0}^n a_k t^k$$

- Cubic polynomial: $p(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

Polynomial

$$p(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n = \sum_{k=0}^n a_k t^k$$

- Cubic polynomial: $p(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

What are the values at $t = 0$ and $t = 1$?

Polynomial

$$p(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n = \sum_{k=0}^n a_k t^k$$

- Cubic polynomial: $p(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

What are the values at $t = 0$ and $t = 1$?

$$p(0) = a_0$$

$$p(1) = a_0 + a_1 + a_2 + a_3$$

Polynomial Splines

Polynomial

$$p(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n = \sum_{k=0}^n a_k t^k$$

- Cubic polynomial: $p(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

What are the values at $t = 0$ and $t = 1$?

$$p(0) = a_0$$

$$p(1) = a_0 + a_1 + a_2 + a_3$$

What are the derivatives with respect to t at $t = 0$ and $t = 1$?

Polynomial

$$p(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n = \sum_{k=0}^n a_k t^k$$

- Cubic polynomial: $p(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

What are the values at $t = 0$ and $t = 1$?

$$p(0) = a_0$$

$$p(1) = a_0 + a_1 + a_2 + a_3$$

What are the derivatives with respect to t at $t = 0$ and $t = 1$?

$$p'(t) = a_1 + 2a_2t + 3a_3t^2$$

$$p'(0) = a_1$$

$$p'(1) = a_1 + 2a_2 + 3a_3$$

$$p''(t) = 2a_2 + 6a_3t$$

$$p''(0) = 2a_2$$

$$p''(1) = 2a_2 + 6a_3$$

$$p'''(t) = 6a_3$$

$$p'''(0) = 6a_3$$

$$p'''(1) = 6a_3$$

Polynomial Splines

Assume we represent the trajectory using polynomial splines.

What is the acceleration cost?

Assume we represent the trajectory using polynomial splines.

What is the acceleration cost?

$$\begin{aligned} J &= \int_{t=0}^1 p''(t) dt \\ &= \int_{t=0}^1 2a_2 + 6a_3 \\ &= 2a_2 t + 3a_3 t^2 \Big|_{t=0}^1 \\ &= 2a_2 + 3a_3 \end{aligned}$$

Polynomial Splines

Let's try to connect 3 numbers $x_1, x_2, x_3 \in \mathbb{R}$ using 2 cubic polynomials (with coefficients a and b):

$$\operatorname{argmin}_{a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3} J_a + J_b \quad \text{s.t.}$$

$$p_a(0) = x_1$$

$$p_a(1) = x_2$$

$$p_b(0) = x_2$$

$$p_b(1) = x_3$$

$$p'_a(1) = p'_b(0)$$

$$p''_a(1) = p''_b(0)$$

Polynomial Splines

Let's try to connect 3 numbers $x_1, x_2, x_3 \in \mathbb{R}$ using 2 cubic polynomials (with coefficients a and b):

$$\underset{a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3}{\operatorname{argmin}} \quad J_a + J_b \quad \text{s.t.}$$

$$p_a(0) = x_1$$

$$p_a(1) = x_2$$

$$p_b(0) = x_2$$

$$p_b(1) = x_3$$

$$p'_a(1) = p'_b(0)$$

$$p''_a(1) = p''_b(0)$$

$$\underset{a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3}{\operatorname{argmin}} \quad 2a_2 + 3a_3 + 2b_2 + 3b_3 \quad \text{s.t.}$$

$$a_0 = x_1$$

$$a_0 + a_1 + a_2 + a_3 = x_2$$

$$b_0 = x_2$$

$$b_0 + b_1 + b_2 + b_3 = x_3$$

$$a_1 + 2a_2 + 3a_3 = b_0$$

$$2a_2 + 6a_3 = 2b_2$$

What kind of optimization problem is that?

What kind of optimization problem is that?

So far LP. (For higher-orders it can become a QP).

What kind of optimization problem is that?

So far LP. (For higher-orders it can become a QP).

How can we handle the 2D or 3D case?

What kind of optimization problem is that?

So far LP. (For higher-orders it can become a QP).

How can we handle the 2D or 3D case?

Optimization can be done for each dimension independently.

Polynomial Splines: Challenges

Poor Numerical Stability

When using high order (≥ 8) and/or many (≥ 50) pieces, it is difficult to solve in practice.

One solution: formulate as unconstrained QP where decision variables are endpoint derivatives of segments [1].

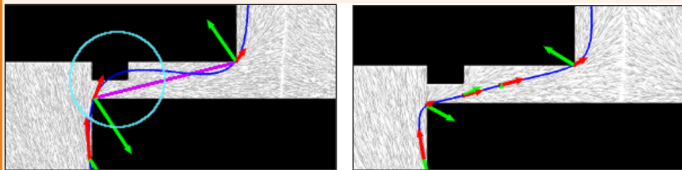
Polynomial Splines: Challenges

Poor Numerical Stability

When using high order (≥ 8) and/or many (≥ 50) pieces, it is difficult to solve in practice.

One solution: formulate as unconstrained QP where decision variables are endpoint derivatives of segments [1].

Handling of Obstacles



- Add additional waypoints

Optimization using Splines

Bézier Curves

Bézier Curve

A Bézier curve $\mathbf{p} : [0, 1] \rightarrow \mathbb{R}^d$ of degree n is defined by $n+1$ control points $\mathbf{p}_0, \dots, \mathbf{p}_n \in \mathbb{R}^d$ as follows:

$$\mathbf{p}(t) = \sum_{i=0}^n b_{i,n}(t) \mathbf{p}_i$$
$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}.$$

Bézier Curve

A Bézier curve $\mathbf{p} : [0, 1] \rightarrow \mathbb{R}^d$ of degree n is defined by $n+1$ control points $\mathbf{p}_0, \dots, \mathbf{p}_n \in \mathbb{R}^d$ as follows:

$$\mathbf{p}(t) = \sum_{i=0}^n b_{i,n}(t) \mathbf{p}_i$$
$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}.$$

Cubic Bézier Curve

$$\begin{aligned} \mathbf{p}(t) = & (1-t)^3 \mathbf{p}_0 + 3t(1-t)^2 \mathbf{p}_1 \\ & + 3t^2(1-t) \mathbf{p}_2 + t^3 \mathbf{p}_3 \end{aligned}$$

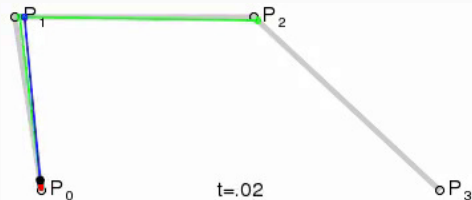
Bézier Curve

A Bézier curve $\mathbf{p} : [0, 1] \rightarrow \mathbb{R}^d$ of degree n is defined by $n+1$ control points $\mathbf{p}_0, \dots, \mathbf{p}_n \in \mathbb{R}^d$ as follows:

$$\mathbf{p}(t) = \sum_{i=0}^n b_{i,n}(t) \mathbf{p}_i$$
$$b_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}.$$

Cubic Bézier Curve

$$\begin{aligned} \mathbf{p}(t) = & (1-t)^3 \mathbf{p}_0 + 3t(1-t)^2 \mathbf{p}_1 \\ & + 3t^2(1-t) \mathbf{p}_2 + t^3 \mathbf{p}_3 \end{aligned}$$



Bézier Curves Properties (1)

- **Endpoint interpolation:** The curve connects $\mathbf{p}_0$ and $\mathbf{p}_n$, i.e., $\mathbf{p}(0) = \mathbf{p}_0$ and $\mathbf{p}(1) = \mathbf{p}_n$

Bézier Curves Properties (1)

- **Endpoint interpolation:** The curve connects $\mathbf{p}_0$ and $\mathbf{p}_n$, i.e., $\mathbf{p}(0) = \mathbf{p}_0$ and $\mathbf{p}(1) = \mathbf{p}_n$
- C^n smoothness

Derivative of Bézier Curve

$$\mathbf{p}'(t) = n \sum_{i=0}^{n-1} b_{i,n-1}(t)(\mathbf{p}_{i+1} - \mathbf{p}_i).$$

Bézier Curves Properties (1)

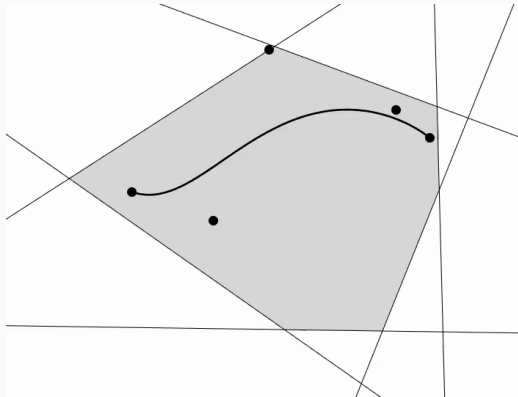
- **Endpoint interpolation:** The curve connects $\mathbf{p}_0$ and $\mathbf{p}_n$, i.e., $\mathbf{p}(0) = \mathbf{p}_0$ and $\mathbf{p}(1) = \mathbf{p}_n$
- C^n smoothness

Derivative of Bézier Curve

$$\mathbf{p}'(t) = n \sum_{i=0}^{n-1} b_{i,n-1}(t)(\mathbf{p}_{i+1} - \mathbf{p}_i).$$

- **Convex hull property:** The curve lies inside the convex hull of their control points, i.e., $\mathbf{p}(t) \in \text{ConvexHull}\{\mathbf{p}_0, \dots, \mathbf{p}_n\} \forall t \in [0, 1]$ [2]

Convex Hull Property



Separating
hyperplane



Safe
polytope

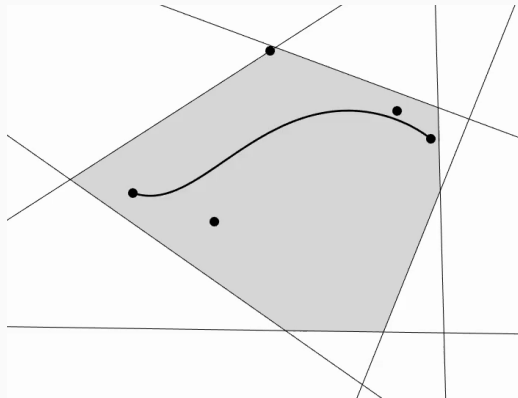


Control
point



Bézier
curve

Convex Hull Property



Separating
hyperplane



Safe
polytope



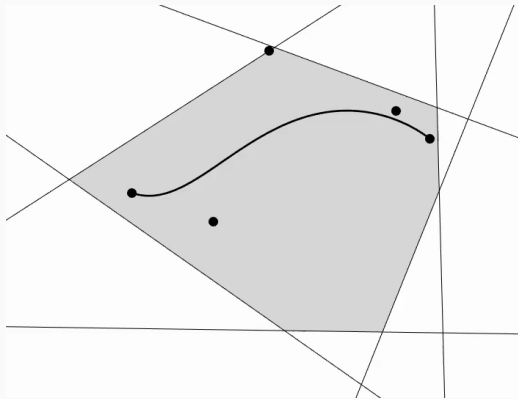
Control
point



Bézier
curve

Why is the convex hull property useful?

Convex Hull Property



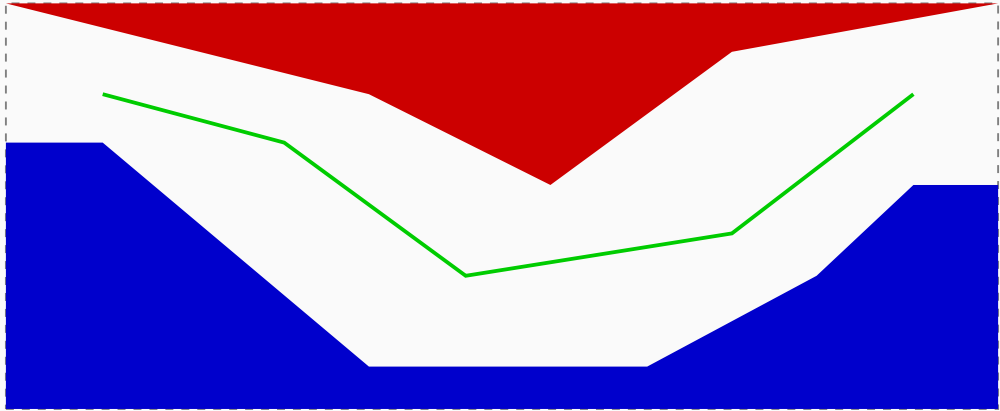
	Separating hyperplane
	Safe polytope
	Control point
	Bézier curve

Why is the convex hull property useful?

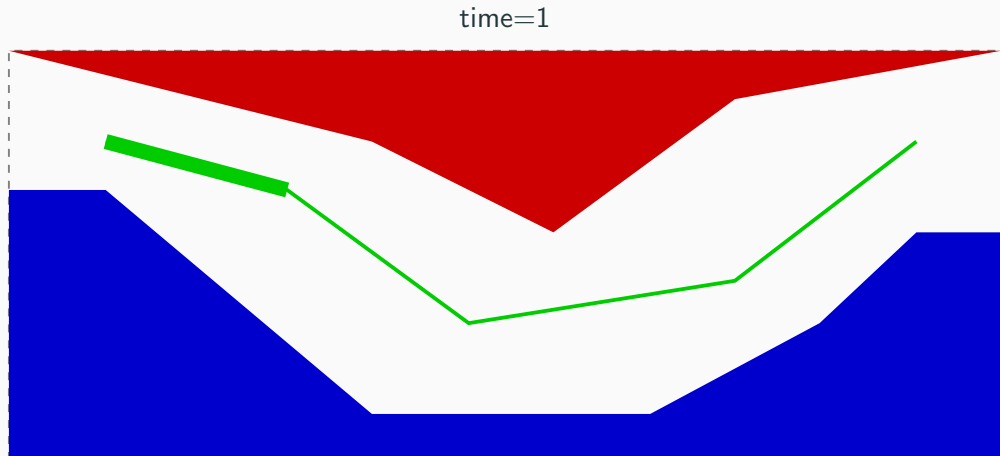
If $\mathbf{p}_i$ are decision variables, we can constrain them to be in $\mathcal{Q}_{free}$. Then, the curve is guaranteed to be collision-free.

Planning in Safe Polyhedra

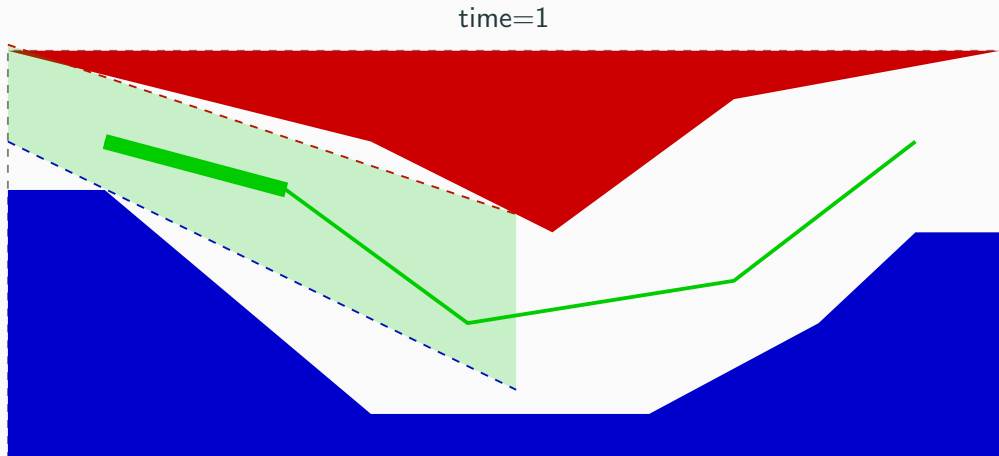
robot path (green), obstacles (blue and red)



Planning in Safe Polyhedra

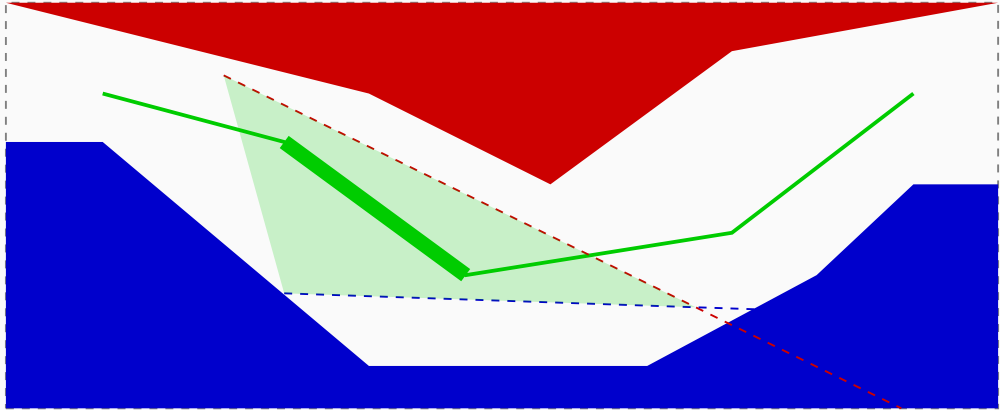


Planning in Safe Polyhedra



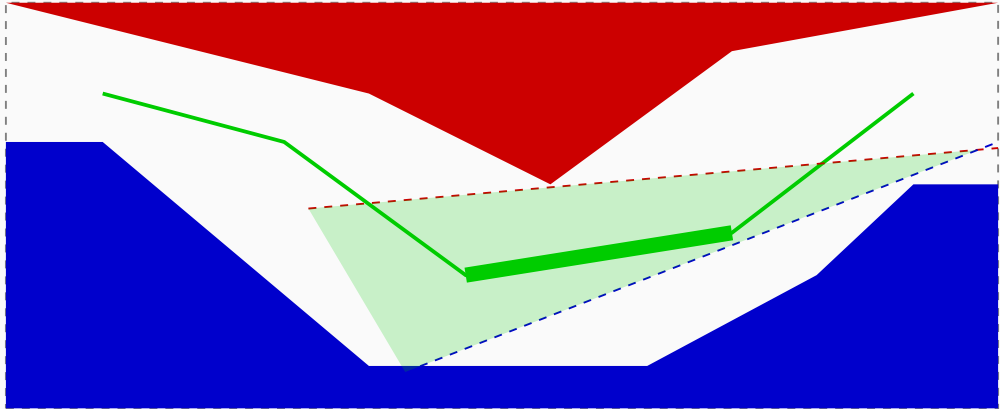
Planning in Safe Polyhedra

time=2



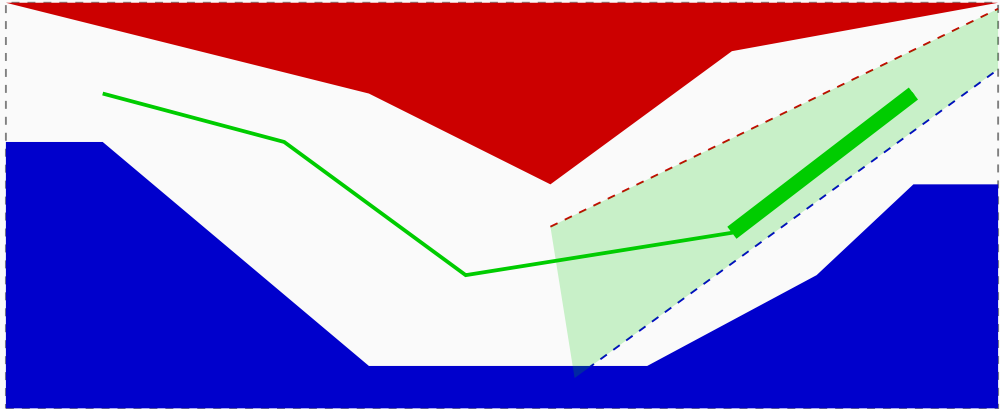
Planning in Safe Polyhedra

time=3



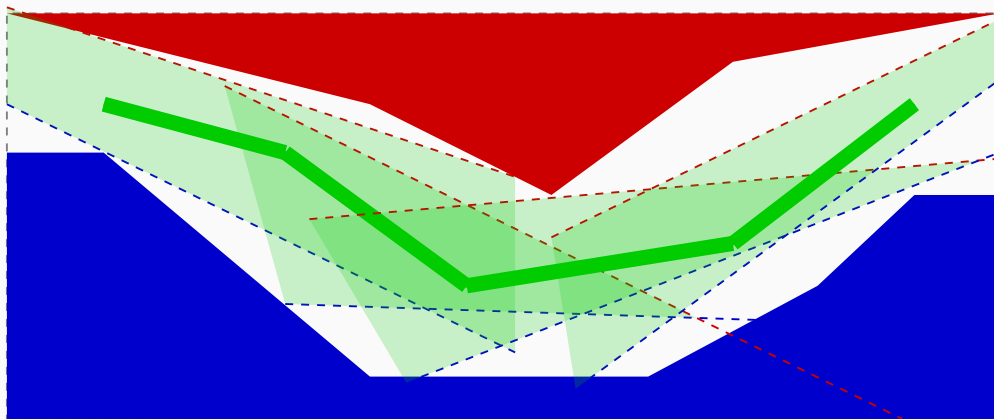
Planning in Safe Polyhedra

time=4

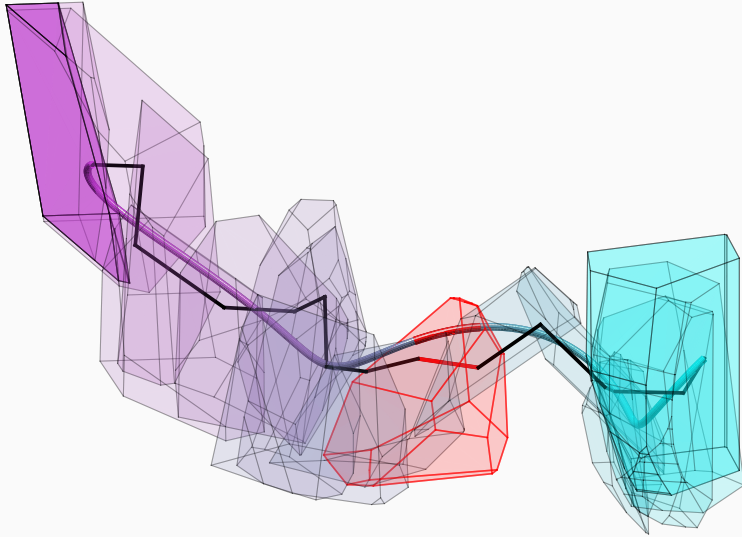


Planning in Safe Polyhedra

robot path (green), obstacles (blue and red)



Planning in Safe Polyhedra (3D Example)

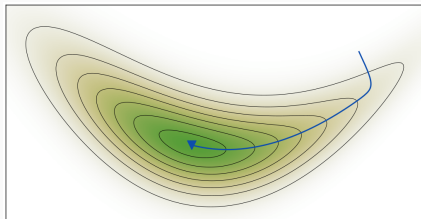


- Optimization for smoothness and energy minimization
- Base splines
- Polynomial splines and convex optimization
- Bézier curves for safe planning

Gradient-based optimization on differentiable costs

Functional Gradients

- Idea: Improve path by deforming it into the direction of a lower cost
- *Functional gradients* as generalization of directions for paths



Functionals

- Define functionals $U : P \rightarrow \mathbb{R}$, with P being the space of functions of the form $p : [0, 1] \rightarrow X$.

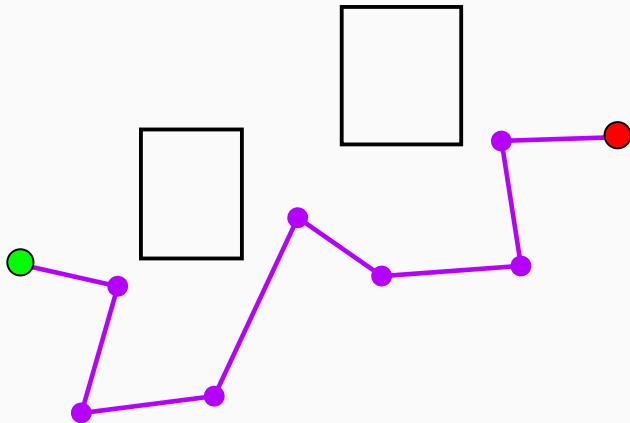
Motion Planning Cost Functionals

Given a path $p : [0, 1] \rightarrow X$, define cost *functionals* like

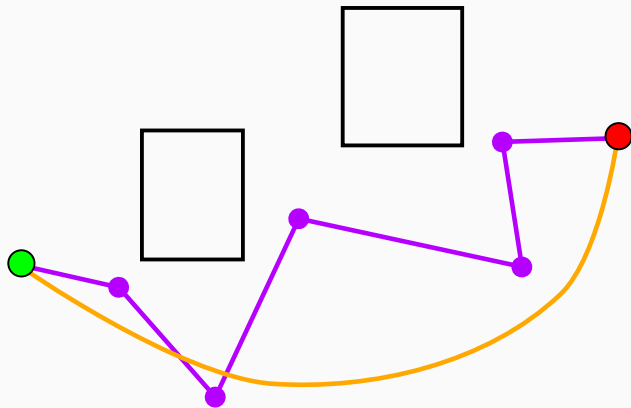
- Obstacle cost functional $U_{obs}[p]$
- Smoothness cost functional $U_{smooth}[p]$
- Path length cost functional $U_{length}[p]$
- Then compute gradients $\nabla U[p]$ and iteratively update paths as

$$p' = p - \lambda \nabla U[p]$$

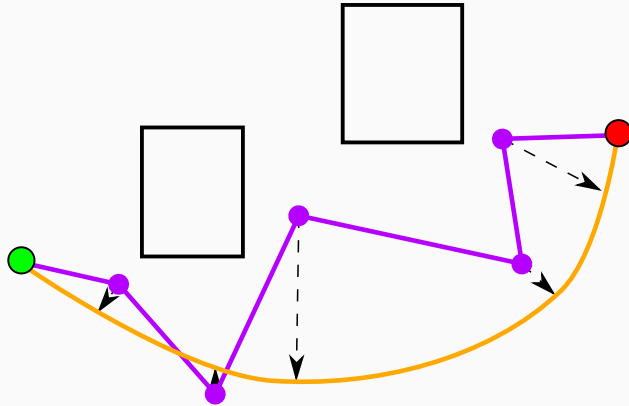
Functional Gradients



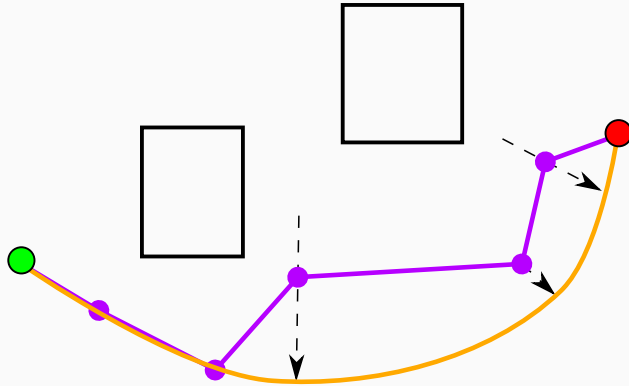
Functional Gradients



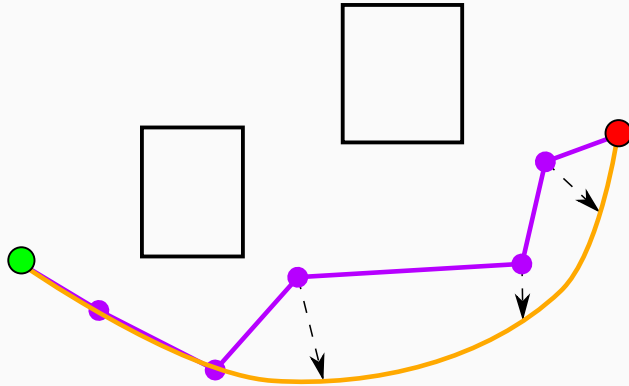
Functional Gradients



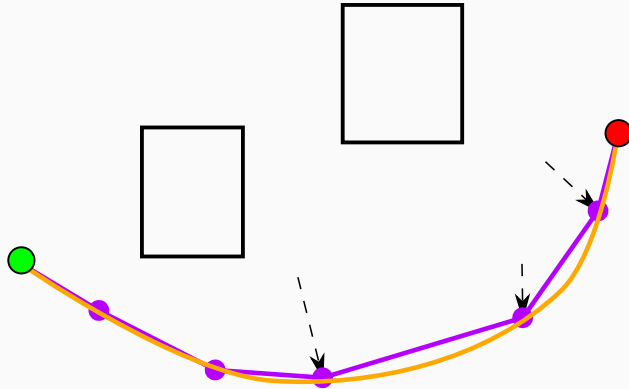
Functional Gradients



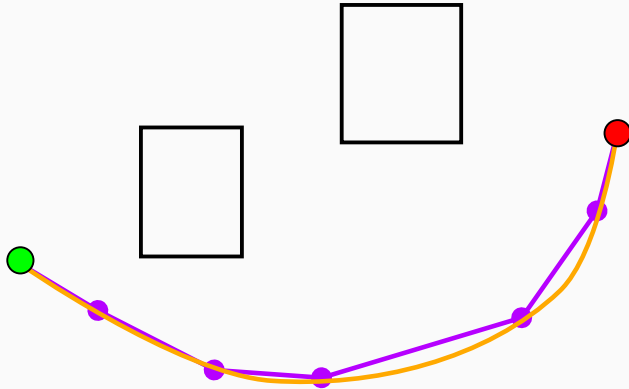
Functional Gradients



Functional Gradients



Functional Gradients



Gradient-based optimization on differentiable costs

Computing Obstacle Cost Gradients

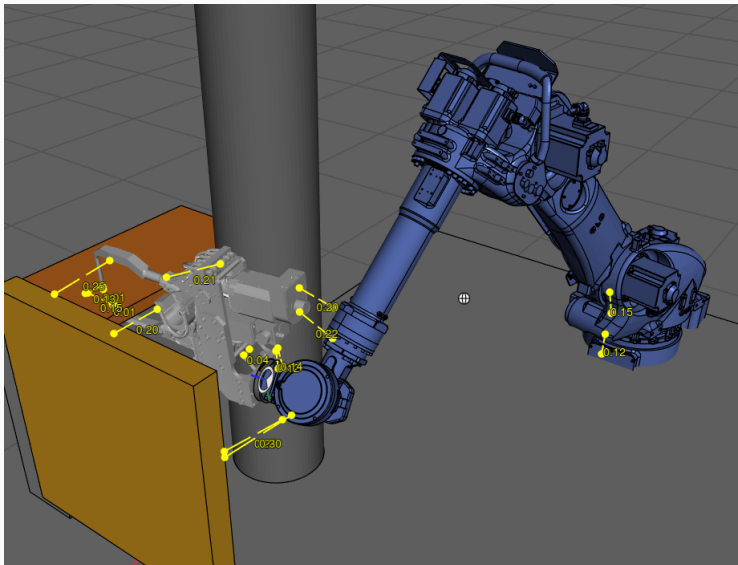
Obstacle cost

- Definition of obstacle cost as clearance from environment
- Obstacle cost

$$U_{obs}[p] = \int_0^1 \int_{x \in B} c_x(p(s)) \, dx \, ds$$

- with x being a prespecified point on the robot body B , c_x being the minimum clearance to the environment and s being the index of the path

Robot point distances



Obstacle cost

$$U_{obs}[p] = \int_0^1 \int_{x \in B} c_x(p(s)) \, dx \, ds$$

Robot clearance

$$c_x(q) = \begin{cases} \frac{1}{\epsilon}(d_x(q) - \epsilon)^2, & \text{if } d_x(q) < \epsilon \\ 0, & \text{otherwise.} \end{cases}$$

whereby $d_x(q)$ is the distance of point x on robot at configuration q to the nearest point in the environment.

Requirements

1. Efficient computation of distances (Signed distance fields)
2. Efficient obstacle cost gradient

Gradient-based optimization on differentiable costs

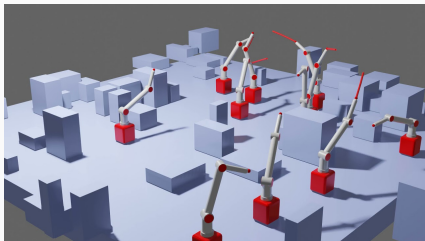
Signed Distance Fields

Obstacle gradients

Problem

Computing distance of a point to the environment is not trivial

- For each point, you need to compute the clearance
- In the worst case, you would need to run GJK once for all obstacles and all points

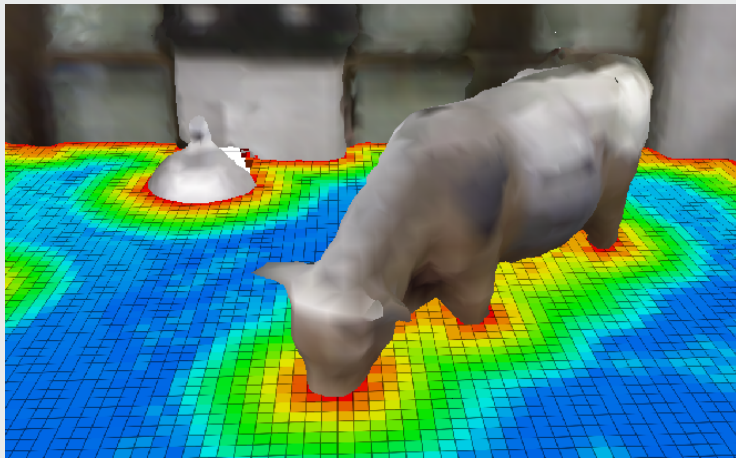


Signed Distance Field

Assumption: All obstacles are static

Then we can precompute a signed distance field (SDF), which assigns for each point in space its clearance value

Signed Distance Field



Can be computed efficiently in 2-d or 3-d.

Signed distance field

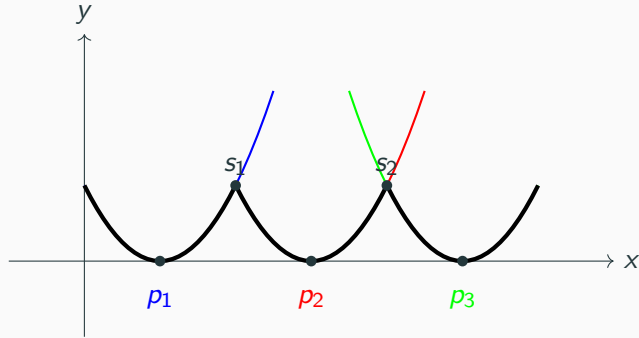
- Voxelize environment
- For each voxel, compute nearest obstacle distance
- Store this information, and use it as a look-up-table to compute clearances

Marching parabola algorithm

Marching parabola algorithm for euclidean distance fields [3] by Felzenszwalb and Huttenlocher, 2012

- Let V be a 1-d grid, and P be occupied voxels on the grid
- Then we search for $D_P(v) = \min_{p \in P} d(v, p)$
- Involves two steps:
- (1) Compute lower envelope parabolas for all occupied voxels
- (2) For each point, check height of lower envelope, and store this value.

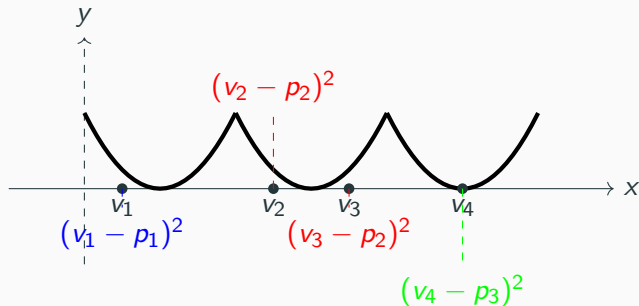
Marching Parabola Algorithm: Building the Lower Envelope



Marching Parabola Algorithm: (1) Building the Lower Envelope

- **Goal:** Find the "lowest curve" formed by a set of parabolas
- **Input:** A set of parabolas, each defined by a point p_i .
- **Process:**
 1. Start with an empty "active list" of parabolas.
 2. For each new parabola:
 - Check where it intersects the last parabola in the list.
 - If it's "too high" (intersects before the current segment), remove the last parabola and repeat.
 - Otherwise, add the new parabola and mark its segment's start point.
 3. Keep track of which parabola is "lowest" for each x-coordinate.
- **Key Idea:** Only keep parabolas that contribute to the lowest curve.

Marching Parabola Algorithm: Assigning Heights



Marching Parabola Algorithm: (2) Assigning Heights

- **Goal:** For each point v (x-coordinate), find its height on the lower envelope.
- **Input:** Points v and the lower envelope from Step 1.
- **Process:**
 1. Start with the first segment of the lower envelope.
 2. For each point v :
 - Move along the envelope until v falls within the current parabola's segment.
 - Compute the height: $\text{height} = (v - \text{parabola's vertex})^2$.
 3. Store the height for each point.

Key Idea: Each point's height comes from the parabola that's lowest at that x-coordinate.

Gradient-based optimization on differentiable costs

2-Dimensional Case

Marching Parabola Algorithm: Handling 2D Grids

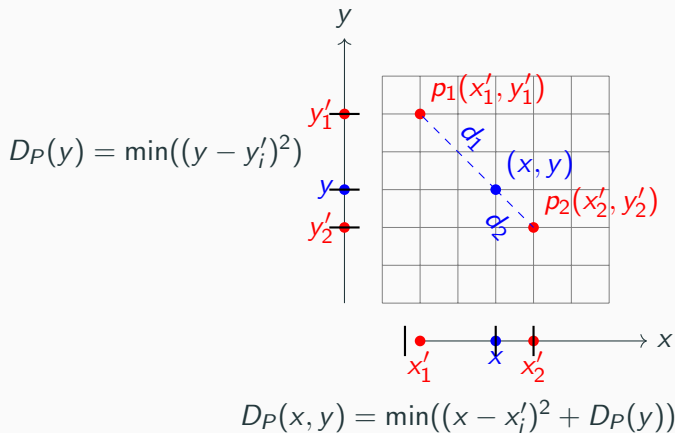
- **Goal:** Find the closest point in set P for each point on a 2D grid.
- **Key Idea:** Solve the 2D problem by breaking it into simpler 1D problems.
- **Process:**
 1. For a grid point (x, y) :
 - Step 1: Find the closest y -coordinate to y among points in P (using the 1D algorithm).
 - Step 2: For each x -coordinate, add the distance $(x - x')^2$ to the result from Step 1.
 - Pick the smallest total distance.
 2. Repeat for all grid points.
- **Complexity:** Scales with grid size (N) and dimensions (d), roughly $d \times N$ steps.

Marching Parabola Algorithm: Handling 2D Grids

- **Goal:** Find the closest point in set P for each point on a 2D grid.
- **Key Idea:** Solve the 2D problem by breaking it into simpler 1D problems.
- **Process:**
 1. For a grid point (x, y) :
 - Step 1: Find the closest y -coordinate to y among points in P (using the 1D algorithm).
 - Step 2: For each x -coordinate, add the distance $(x - x')^2$ to the result from Step 1.
 - Pick the smallest total distance.
 2. Repeat for all grid points.
- **Complexity:** Scales with grid size (N) and dimensions (d), roughly $d \times N$ steps.

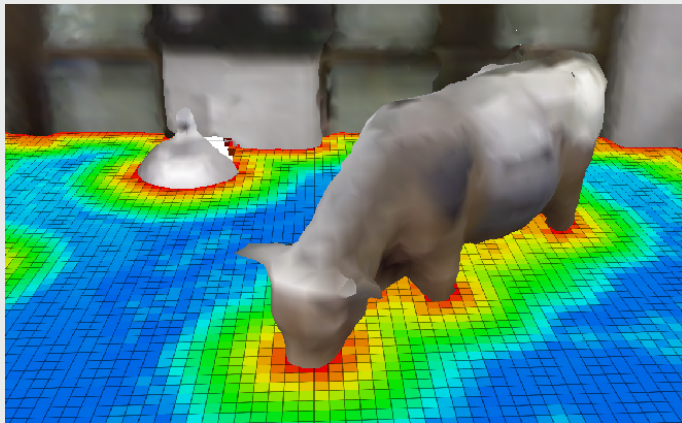
What would be the complexity for a naive algorithm with grid size N and M occupied voxels?

Marching Parabola Algorithm: Handling 2D Grids



Source: Felzenszwalb, P.F., Huttenlocher, D.P., "Distance transforms of sampled functions," Theory of Computing, 2012.

Signed Distance Field



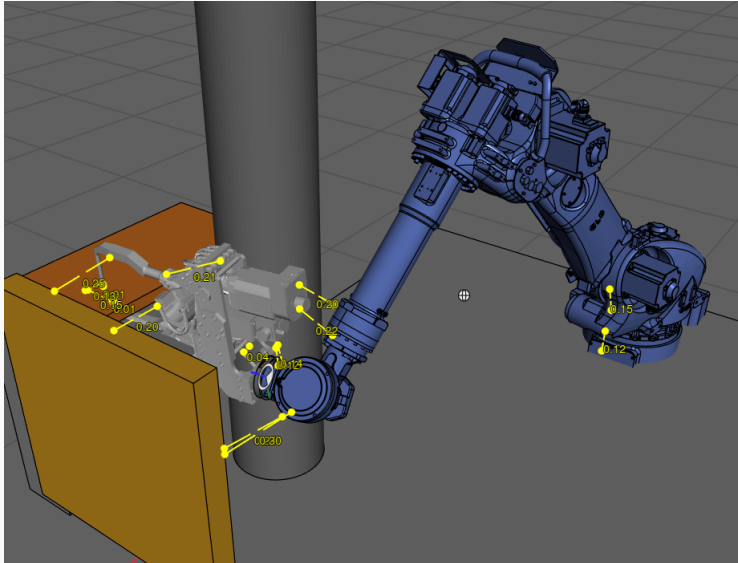
How can this be useful for optimization of non-point robots?

Gradient-based optimization on differentiable costs

Obstacle cost gradient

- Given $U_{obs}[p]$, let us compute $\nabla U_{obs}[p]$.
- Three steps
 - Compute gradient of clearance
 - Consider gradient as virtual forces
 - Map forces into configuration space

Robot point distances



Step 1: Compute gradient of clearance

- Clearance $c_x(q)$ will be improved by moving point x away from obstacle
- Let n_E be the nearest point on the environment to x .
- Then

$$\frac{x - n_E}{\|x - n_E\|}$$

Step 2: Consider gradient as virtual forces

- Pushing at each x along the direction of $\frac{x - n_E}{\|x - n_E\|}$ would increase clearance!
- Let us define virtual forces at each x and map them into the configuration space

Step 3: Map forces into configuration space

- A virtual force will move each point x by an infinitesimal small dx .
- This will create likewise an infinitesimal small dq in configuration space
- A dq is the direction in configuration space, which (locally) increases clearance.

Step 3: Map forces into configuration space

- How to do it? By taking the derivative of the forward kinematics.
- Let x be a point in the workspace, q be the joint space values, and T the forward kinematics.

$$\begin{aligned}x &= T(q) \\ \dot{x} &= \frac{dT(q)}{dq} \dot{q} \\ &= J \cdot \dot{q}\end{aligned}$$

- Taking the inverse leads to $\dot{q} = J^{-1}(q)\dot{x}$ (transpose, pseudoinverse)

Steepest Descent Step

- Functional gradient

$$\nabla U[p] = \int_0^1 \sum_{x \in R(p(s))} J^{-1}(p(s)) \nabla c_x p(s) \, ds$$

- Gradient descent step in functional space:

$$p' = p - \lambda \nabla U[p]$$

Functional Gradient Descent

```
Let p be a path  
While not converged  
     $p' = p - \text{lambda} * dU[p]$   
Return p'
```

Summary of obstacle cost gradient

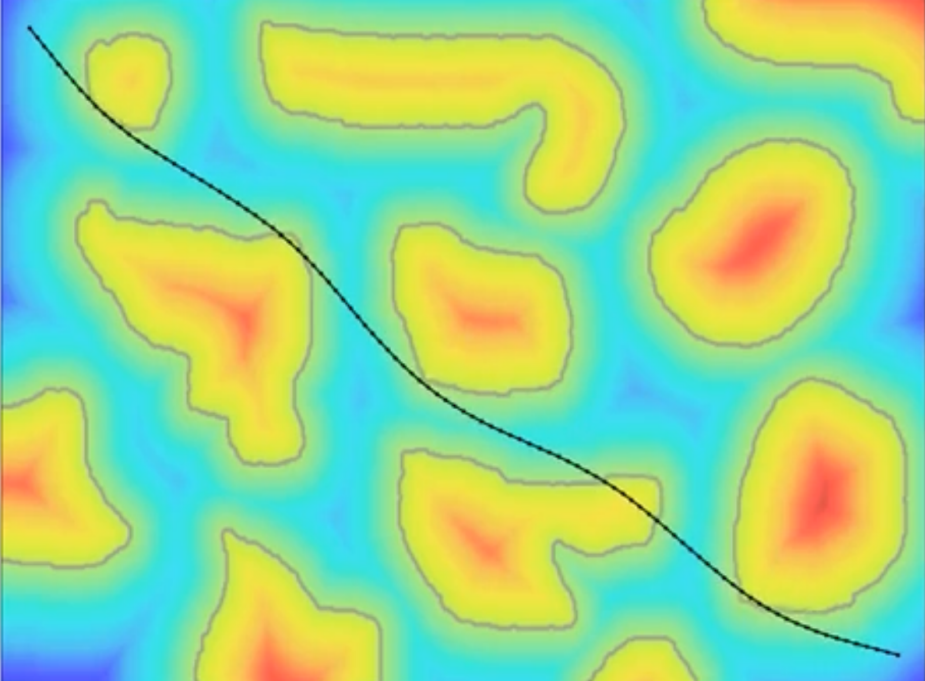
- (1) Formulate cost from distances
- (2) Compute virtual displacements in direction of decreasing cost
- (3) Map virtual displacements into configuration space using jacobian
- (4) Take those displacements as directions for gradient step
- (5) Apply gradient descent using the directional steps in configuration space

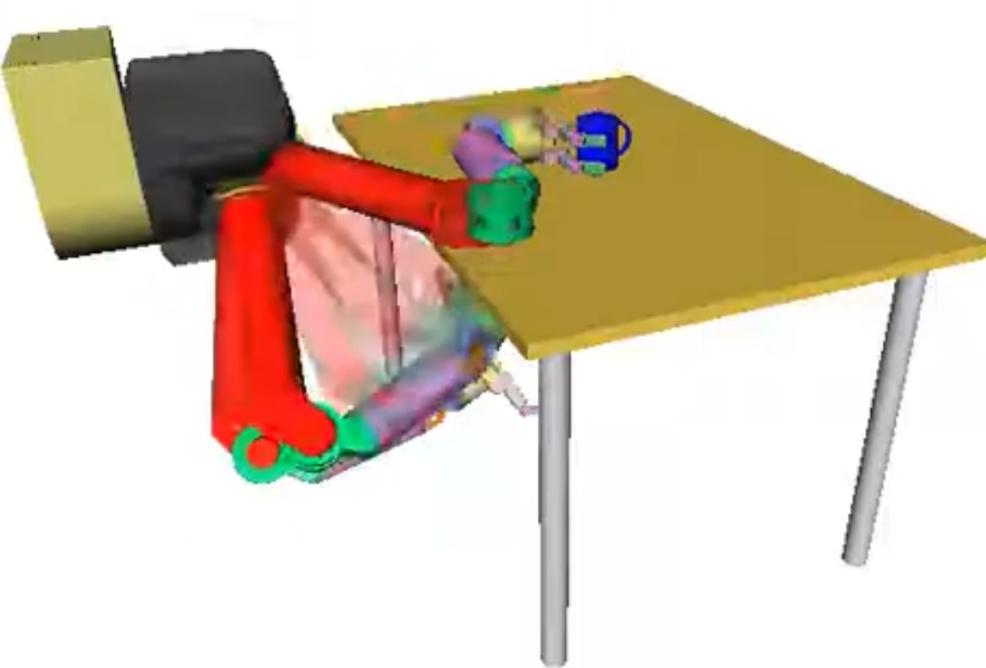
CHOMP

Covariant Hamiltonian Optimization for Motion Planning (CHOMP)

- Initialize a path
- Discretize the path (waypoints)
- Use cost functional $U[p] = U_{obs}[p] + \lambda U_{smooth}[p]$
- Apply derivative to waypoints
- Repeat or terminate if magnitude of gradient falls below threshold

M Zucker et al., "Chomp: Covariant hamiltonian optimization for motion planning", 2013 [4]





Why can CHOMP move paths out of collision?

Why can CHOMP move paths out of collision?

Are the paths optimal?

Why can CHOMP move paths out of collision?

Are the paths optimal?

Is this complete?

Summary

- Optimization, gradient-free vs. gradient-based
- Shortcutting
- B-splines
- Polynomial splines (acceleration optimized)
- Bézier curves (obstacle-free)
- Computing obstacle costs (Signed distance field, marching parabolas, obstacle gradients)
- Functional gradient descent and CHOMP

- [1] Charles Richter, Adam Bry, and Nicholas Roy. **“Polynomial Trajectory Planning for Aggressive Quadrotor Flight in Dense Indoor Environments”**. In: *International Symposium on Robotics Research (ISRR)*. Vol. 114. Springer Tracts in Advanced Robotics. Springer, 2013, pp. 649–666. DOI: 10.1007/978-3-319-28872-7_37.
- [2] Rida T. Farouki. **“The Bernstein polynomial basis: A centennial retrospective”**. In: *Computer Aided Geometric Design* 29.6 (2012), pp. 379–419. DOI: 10.1016/j.cagd.2012.03.001.
- [3] Pedro F Felzenszwalb and Daniel P Huttenlocher. **“Distance transforms of sampled functions”**. In: *Theory of computing* 8.1 (2012), pp. 415–428.

- [4] Matt Zucker, Nathan Ratliff, Anca D Dragan, Mihail Pivtoraiko, Matthew Klingensmith, Christopher M Dellin, J Andrew Bagnell, and Siddhartha S Srinivasa. **“Chomp: Covariant hamiltonian optimization for motion planning”**. In: *The International Journal of Robotics Research* 32.9-10 (2013), pp. 1164–1193.